

1. Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.

Run SQL query/queries on database restaurant: 

```
1 CREATE TABLE Restaurant (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     location VARCHAR(100) NOT NULL,  
5     cuisine VARCHAR(50)  
6 );
```

 Browse  Structure  SQL  Search  Insert  Export  Import

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0019 seconds.)

SELECT * FROM `restaurant`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

id name location cuisine

Query results operations

 Create view

2. Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).

Run SQL query/queries on database restaurant:

```
1 CREATE TABLE FoodOrder (  
2     order_id INT PRIMARY KEY,  
3     restaurant_id INT,  
4     user_id INT,  
5     order_total DECIMAL(10,2),  
6     FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
7 );
```

Browse Structure SQL Search Insert Export Import

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

SELECT * FROM `foodorder`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

order_id	restaurant_id	user_id	order_total
----------	---------------	---------	-------------

Query results operations

3. Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.
Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

Structure
 SQL
 Search
 Query
 Export
 Import

Run SQL query/queries on database restaurant:

```

1 CREATE TABLE SpotifyUser (
2     user_id INT PRIMARY KEY,
3     username VARCHAR(50) UNIQUE,
4     email VARCHAR(100) NOT NULL,
5     subscription_type VARCHAR(30)
6 );
7
8 INSERT INTO SpotifyUser VALUES
9 (1, 'Aarav', 'aarav@gmail.com', 'Premium'),
10 (2, 'Riya', 'riya@gmail.com', 'Free'),
11 (3, 'Neha', 'neha@gmail.com', 'Premium'),
12 (4, 'Karan', 'karan@gmail.com', 'Premium'),
13 (5, 'Meera', 'meera@gmail.com', 'Free'),
14 (6, 'Rahul', 'rahul@gmail.com', 'Premium');

```

Extra options

				user_id	username	email	subscription_type
☐				1	Aarav	aarav@gmail.com	Premium
☐				2	Riya	riya@gmail.com	Free
☐				3	Neha	neha@gmail.com	Premium
☐				4	Karan	karan@gmail.com	Premium
☐				5	Meera	meera@gmail.com	Free
☐				6	Rahul	rahul@gmail.com	Premium

☐ Check all
 With selected:
 Edit
 Copy
 Delete
 Export

4. Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

Run SQL query/queries on table restaurant.spotifyuser: ?

```
1 CREATE VIEW TopSpendersView AS
2 SELECT SpotifyUser.username, FoodOrder.order_total
3 FROM SpotifyUser
4 JOIN FoodOrder
5 ON SpotifyUser.user_id = FoodOrder.user_id
6 WHERE FoodOrder.order_total > 1000;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0027 seconds.)

CREATE VIEW TopSpendersView AS SELECT SpotifyUser.username, FoodOrder.order_total FROM SpotifyUser JOIN FoodOrder ON SpotifyUser.user_id = FoodOrder.user_id WHERE FoodOrder.order_total > 1000;

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

5. Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.
Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.

Run SQL query/queries on table restaurant.spotifyuser: ?

```
1 SELECT username, order_total
2 FROM TopSpendersView
3 WHERE order_total > 2000
4 ORDER BY order_total DESC;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)

SELECT username, order_total FROM TopSpendersView WHERE order_total > 2000 ORDER BY order_total DESC;

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

